

VIBESMEET / OPEN CONCERT

Demand-Driven Gig Creation MVP

Corrected source package, Facebook integration, technical README, validation report, and UI screenshots

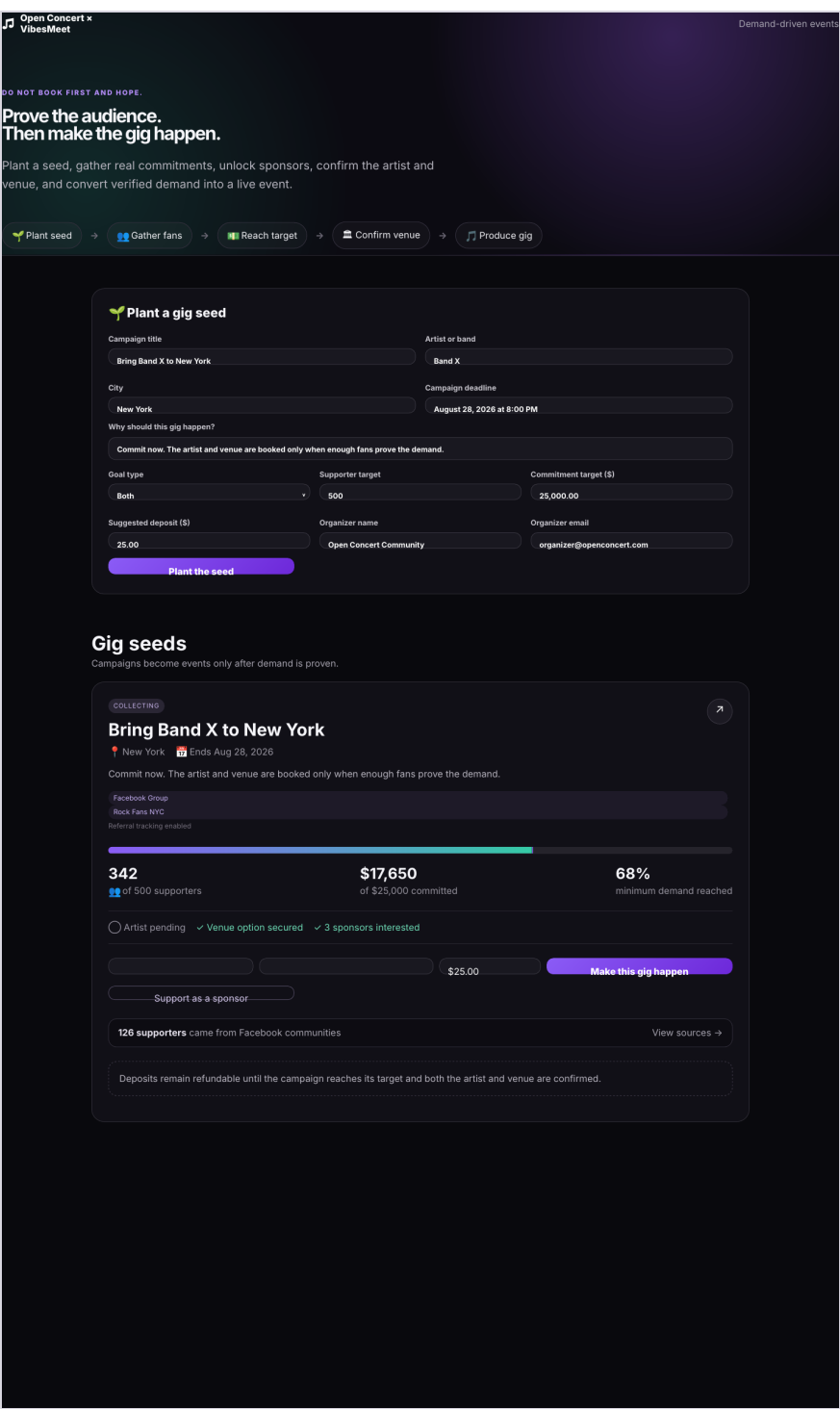
A production-minded reference implementation for proving audience demand before booking an artist or venue.

Plant the seed → Gather supporters → Reach the target → Confirm artist and venue → Produce the gig

Validated July 29, 2026

Product Screenshots

1. Landing Page and Plant a Gig Seed



Desktop landing page, campaign discovery, and demand-campaign creation flow.

1. Campaign Detail

Open Concert x VibesMeet

Share campaign

Gig seeds / New York / Bring Band X

COLLECTING

Bring Band X to New York

A fan-powered concert campaign. No artist or venue is financially locked until verified demand reaches the published minimum.

New York, NY Campaign closes August 28 Started by Open Concert Community

412
of 500 supporters

\$21,250
of \$25,000 committed

\$4,500
sponsor commitments

82%
minimum demand reached

88 more fans can unlock the booking process.
Once the threshold is reached, the organizer has 14 days to confirm the artist and venue. Deposits remain refundable until both confirmations are complete.

Make this gig happen

Your name

Email address

1 ticket commitment

\$25 refundable deposit

Support this gig with \$25

Your card is authorized now. The deposit becomes final only after the audience target, artist confirmation, and venue confirmation are complete.

Booking conditions

Audience threshold82% reached

Artist availabilityAwaiting threshold

Venue option✓ Tentatively held

Sponsor coverage✓ \$4,500 committed

Sponsor this audience

Local Sound Co.

Contact name

Business email

\$1,000.00

Logo placement and VIP activation area

Commit sponsorship

Audience sources

Facebook groups154

Direct sharing117

Creator profiles89

Sponsor referrals52

Top communities

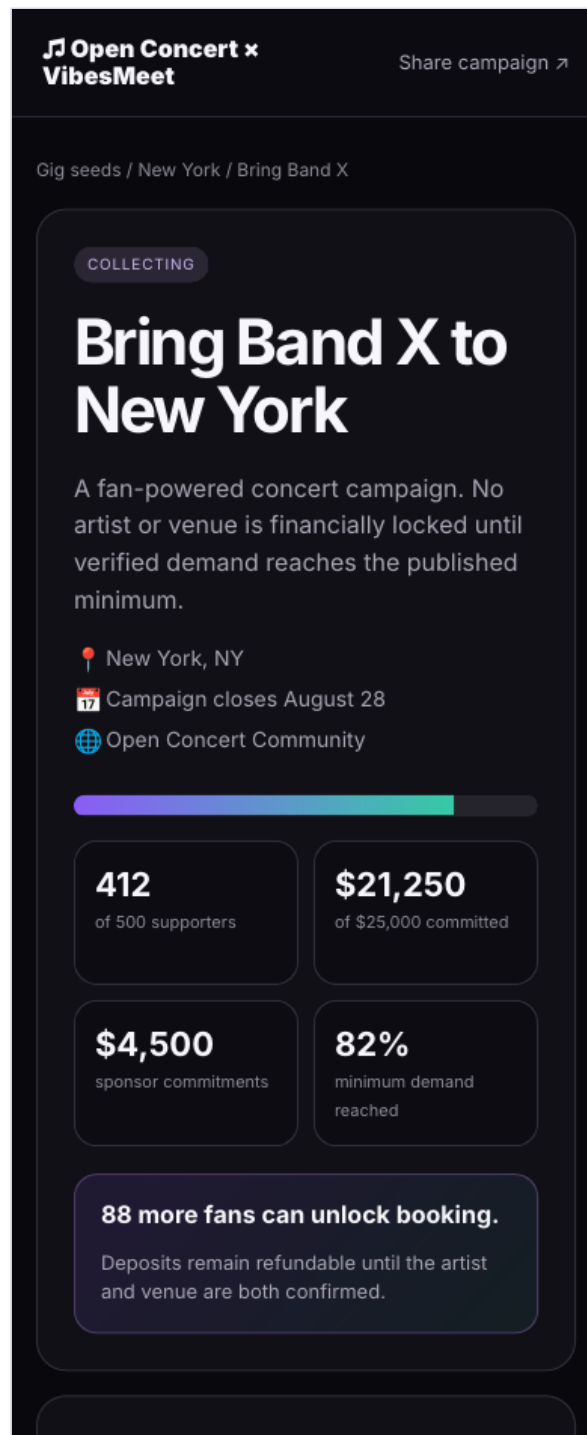
Rock Fans NYC80 fans

Live Music Brooklyn41 fans

Band X Global27 fans

Campaign progress, supporter and financial thresholds, refundable commitments, artist and venue confirmation, sponsorship, and Facebook-source attribution.

1. Mobile Campaign - Top



Top section of the mobile campaign experience.

1. Mobile Campaign - Middle

Make this gig happen

Your card is authorized now. The deposit becomes final only after every published condition is met.

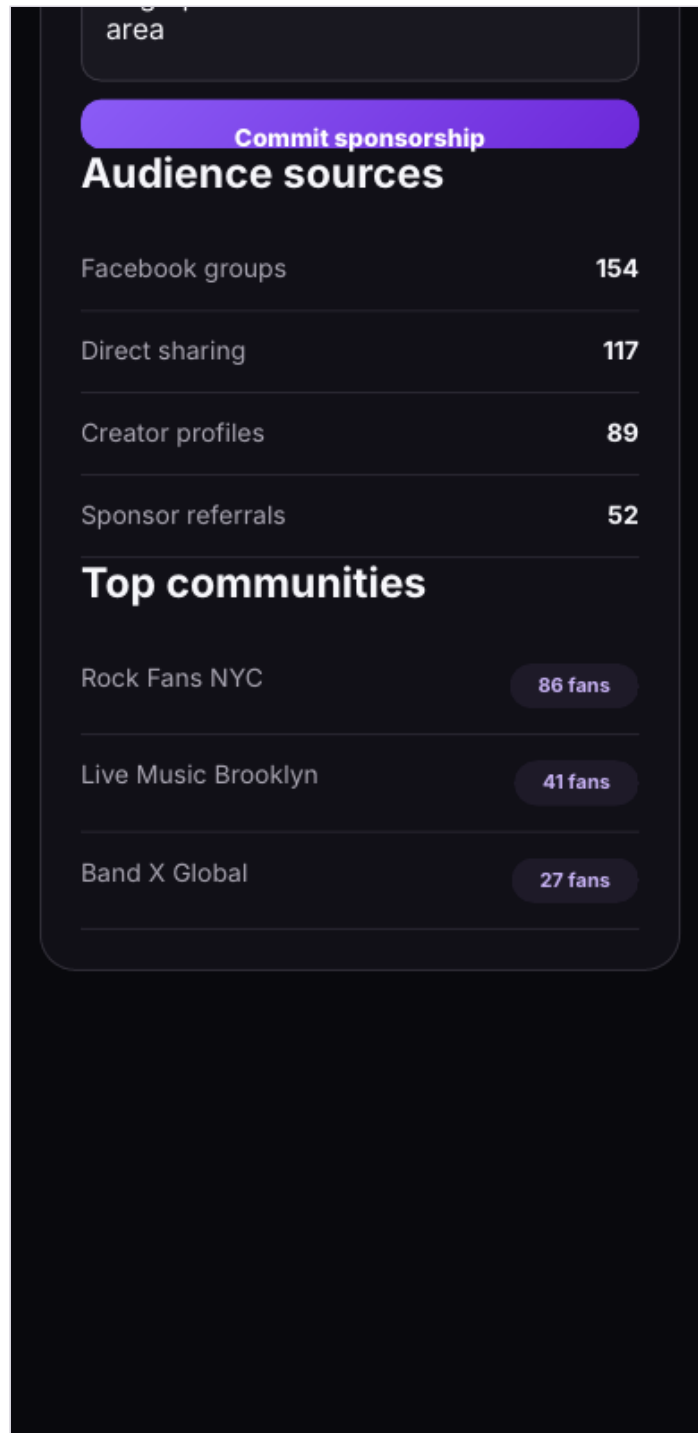
Booking conditions

Audience threshold	82% reached
Artist availability	Awaiting threshold
Venue option	✓ Tentatively held
Sponsor coverage	✓ \$4,500 committed

Sponsor this audience

Middle section of the mobile campaign experience.

1. Mobile Campaign - Bottom



Bottom section of the mobile campaign experience.

Demand-Driven Gig Creation MVP

A production-minded reference implementation of the missing Open Concert / VibesMeet workflow:

Plant the seed → gather supporters → reach the target → confirm the artist and venue → produce the gig.

This is not ordinary ticketing. A campaign proves real demand before an organizer assumes the full cost and risk of booking the artist, venue, production, security, insurance, travel, and promotion.

Product idea

A fan, artist, promoter, venue, sponsor, or community administrator proposes a gig such as:

Bring Band X to New York.

The organizer defines one or both minimum thresholds:

- 500 verified supporters
- \$25,000 in refundable commitments and sponsorships

Fans can register their intent, reserve a place, or pay a refundable deposit. Sponsors can commit after seeing verified audience demand. Only after the threshold is reached does the organizer confirm the artist and venue and convert the campaign into a normal ticketed VibesMeet event.

When the threshold is missed or the artist and venue cannot be confirmed under the published terms, deposits are refunded.

Included

Demand-campaign platform

- Django REST Framework API
- PostgreSQL-ready data model with SQLite fallback
- React + TypeScript + Vite interface
- Supporter attendance commitments
- Refundable supporter deposits
- Sponsor commitments
- Supporter-count, committed-money, or combined thresholds
- Atomic threshold evaluation to prevent race conditions
- Artist and venue confirmation gates
- Conversion into a live VibesMeet event through an `event_id`
- Failed-campaign refund workflow
- Stripe adapter and local fake-payment adapter
- Stripe webhook verification
- Idempotency keys for pledge submissions

- Immutable campaign-event audit log
- Automated tests for success and refund flows
- Docker Compose setup

Facebook-first organizer integration

- Existing Facebook Event URL attached to every gig seed
- Existing Facebook Group URL attached to every gig seed
- Artist, venue, or organizer Facebook Page URL
- Facebook Share Dialog with campaign-specific links
- Open Graph preview pages optimized for Facebook sharing
- Facebook Login with backend token validation
- Discovery of Facebook Pages managed by the connected organizer
- Publishing a campaign link to a managed Facebook Page
- Unique Group, community, and administrator referral codes
- Attribution of supporters, deposits, and revenue to each Facebook source
- Meta Pixel browser events
- Meta Conversions API server events
- Browser/server event IDs for deduplication
- Website, app, and physical attendance/check-in conversion events
- Facebook source data in the immutable campaign audit log

Facebook integration strategy

The operating principle

Event organizers already use **Facebook Events, Facebook Groups, Facebook Pages, Messenger, and community discussions** as their primary discovery and promotion channels.

The goal is not to force organizers to abandon those communities. The integration uses Facebook as the audience and communication layer while VibesMeet becomes the transaction, verification, sponsorship, and fulfillment layer.

```
Facebook Group / Event / Page
  ↓
Tracked "Make This Gig Happen" link
  ↓
VibesMeet demand campaign
  ↓
Verified supporter + refundable commitment
  ↓
Threshold reached
  ↓
Artist + venue confirmed
  ↓
Normal VibesMeet ticketed event
  ↓
Facebook Page/Event promotion + measured attendance
```


Why full Facebook Group syncing is not implemented

Meta retired the general-purpose Facebook Groups API and its permissions, including `publish_to_groups` and `groups_access_member_info`, for all versions on April 22, 2024.

Therefore, a compliant modern application cannot:

- Import a Facebook Group membership list
- Read private Group member data
- Automatically publish into arbitrary Groups
- Synchronize all Group posts and comments
- Automatically invite every Group member

This project implements the strongest supported alternative:

1. A Group administrator creates a unique tracked link.
2. The administrator manually posts or pins it in the Group or Facebook Event discussion.
3. Facebook generates a rich Open Graph preview.
4. Every visitor carries `source`, `group`, and `ref` attribution into VibesMeet.
5. VibesMeet measures supporters, deposits, sponsorships, conversions, and eventual attendance from that community.

Official Meta reference:

- <https://developers.facebook.com/docs/graph-api/changelog/version19.0/>

Recommended organizer workflow

Phase 1 — Plant the seed

The organizer creates the demand campaign and optionally attaches:

- Existing Facebook Event URL
- Main Facebook Group URL
- Artist, venue, or organizer Facebook Page URL
- Minimum supporter target
- Minimum financial target
- Suggested refundable deposit
- Campaign deadline

The Facebook Event can remain the familiar public discussion and discovery hub. The VibesMeet campaign becomes the authoritative place for verified demand.

Phase 2 — Connect each Facebook community

For each Facebook Group, Event discussion, Page, influencer, artist fan club, or promoter, generate a separate link:

```
https://your-domain.example/share/campaign/bring-band-x-new-york/  
?source=facebook_group  
&group=Band%20X%20NYC%20Fans  
&ref=admin-jane
```

Examples:

```
source=facebook_group&group=Band X NYC Fans&ref=admin-jane  
source=facebook_event&group=Official NYC Event&ref=event-discussion  
source=facebook_page&group=Band X Official&ref=page-organic  
source=facebook_ads&group=NYC Rock Audience&ref=adset-2026-07  
source=facebook_messenger&group=Street Team&ref=messenger-captain-4
```

This enables community-level reporting without copying Facebook member data.

Phase 3 — Share and pin

The organizer should place the tracked campaign URL in:

- Facebook Event ticket or external-link field where available
- Facebook Event description
- Event discussion pinned post
- Facebook Group pinned or featured post
- Facebook Page post
- Messenger community chats
- Artist and venue Page posts
- Relevant comments, with administrator approval
- Meta advertising campaigns

Suggested Facebook post:

Should Band X play New York?

The show is not booked yet. We are proving demand first.

Support the campaign, reserve your place, or leave a refundable deposit. Once the minimum audience and funding target is reached, the artist and venue will be confirmed and the gig will happen.

Make this gig happen:
<TRACKED_CAMPAIGN_LINK>

Phase 4 — Measure demand

Every pledge records:

- source
- source_label
- referral_code
- Supporter count
- Commitment amount

- Payment status
- Campaign and timestamp

Example pledge payload:

```
{
  "supporter_name": "Alex Fan",
  "supporter_email": "alex@example.com",
  "quantity": 1,
  "amount": "25.00",
  "idempotency_key": "browser-generated-uuid",
  "source": "facebook_group",
  "source_label": "Band X NYC Fans",
  "referral_code": "admin-jane"
}
```

This lets the organizer answer:

- Which Facebook Group produced the most supporters?
- Which Page generated the most deposits?
- Which administrator or promoter has the best conversion rate?
- Which geographic audience should receive the confirmed event announcement?
- Which community should receive a referral fee, recognition, or ticket allocation?

Phase 5 — Reach the threshold

When the configured minimum is reached, the campaign moves atomically from:

COLLECTING → TARGET_REACHED

The organizer then begins artist and venue confirmation. Supporter commitments are not silently treated as final ticket purchases before the published conditions are satisfied.

Phase 6 — Confirm artist and venue

The campaign records separate confirmation gates:

Artist confirmed: yes/no
Venue confirmed: yes/no

Both must be confirmed before the campaign becomes a live event.

Recommended production additions:

- Artist letter of intent
- Venue hold expiration date
- Capacity and accessibility
- Production rider
- Insurance
- Permit requirements
- Cancellation terms

- Deposit conversion terms

Phase 7 — Convert to the normal VibesMeet event

After artist and venue confirmation:

```
POST /api/campaigns/<slug>/finalize/
```

The Demand Campaign service publishes `campaign.finalized`. VibesMeet Event OS creates the ordinary confirmed event and returns its `event_id`.

At this point:

- Eligible deposits become final according to campaign terms
- Ticket inventory opens
- The event receives its standard ticketing page
- Facebook Event and Page posts can be updated with the confirmed date, venue, and ticket URL
- SponsorOS receives verified demand and source-attribution data

Phase 8 — Promote and measure attendance

Use both Meta Pixel and Conversions API to send funnel events such as:

```
PageView
ViewContent
Lead
InitiateCheckout
Purchase
AttendEvent          # custom physical check-in event
CampaignRefunded     # custom lifecycle event
```

For physical QR check-in, send a server event with:

```
{
  "event_name": "AttendEvent",
  "event_id": "checkin-vibes-event-123-ticket-456",
  "email": "fan@example.com",
  "action_source": "physical_store",
  "custom_data": {
    "event_id": "vibes-event-123",
    "ticket_id": "ticket-456",
    "venue": "Example Hall"
  }
}
```

API:

```
POST /api/campaigns/<slug>/facebook/track-conversion/
```

This gives organizers a complete measurement chain from Facebook discovery through real-world attendance.

Facebook integration matrix

Integration	Implemented	Notes
Existing Facebook Event URL	Yes	Stored with the campaign and displayed in the UI
Existing Facebook Group URL	Yes	Stored as the principal community hub
Facebook Page URL	Yes	Stored for the artist, venue, or organizer
Facebook Share Dialog	Yes	Opens a tracked campaign URL
Rich Facebook preview	Yes	Dynamic Open Graph page per campaign/source
Facebook Login	Yes	JavaScript SDK plus backend token validation
Managed Page discovery	Yes	Uses /me/accounts after permission approval
Publish campaign to Page	Yes	Uses /{page-id}/feed
Meta Pixel	Yes	Browser PageView and funnel events
Meta Conversions API	Yes	Server-side commitment, sponsor, purchase, and custom events
Browser/server deduplication	Yes	Shared event_id convention
Group member import	No	Groups API removed by Meta
Automatic arbitrary Group posting	No	Groups API removed by Meta
Automatic Group invitations	No	Not available through the current supported API
Existing Facebook Event creation/update	Manual workflow	Keep the Event as discovery hub and link it to the VibesMeet campaign
Messenger sharing	Link copy/share	Tracked URLs can be pasted into Messenger chats
Instagram promotion	Campaign URL	Use the same source/referral model for bio, stories, ads, and DMs
WhatsApp promotion	Campaign URL	Use tracked links for chats and communities
Offline attendance attribution	Yes	Custom Conversions API event with physical_store action source

Facebook App setup

1. Create a Meta app

Create a Meta developer application and add the relevant products:

- Facebook Login
- Marketing API / Conversions API as required by the selected business setup

Official documentation:

- <https://developers.facebook.com/docs/facebook-login/web/>
- <https://developers.facebook.com/documentation/ads-commerce/conversions-api>
- <https://developers.facebook.com/documentation/meta-pixel>

2. Configure OAuth

Add the production frontend domain and valid OAuth redirect URIs in the Meta application dashboard.

For local development, the frontend is normally:

```
http://localhost:5173
```

Production must use HTTPS.

3. Request Page permissions

The Page publishing workflow requests:

```
public_profile
email
pages_show_list
pages_read_engagement
pages_manage_posts
```

Depending on the app mode, organization, users, and intended public use, Meta may require App Review, a screencast, test credentials, business verification, and a clear explanation of how each permission is used.

The production application should request only the permissions actually needed.

Official Page publishing reference:

- <https://developers.facebook.com/docs/graph-api/reference/page/feed/>

4. Configure environment variables

Backend .env:

```
PUBLIC_BASE_URL=https://api.your-domain.example
META_GRAPH_API_VERSION=v25.0
META_APP_ID=your-meta-app-id
META_APP_SECRET=your-meta-app-secret
META_PIXEL_ID=your-pixel-id
META_CONVERSIONS_API_TOKEN=your-system-user-or-dataset-token
META_TEST_EVENT_CODE=
META_DEFAULT_SHARE_IMAGE=https://cdn.your-domain.example/default-gig-share.jpg
```

Frontend .env:

```
VITE_API_BASE=https://api.your-domain.example/api
VITE_STRIPE_PUBLISHABLE_KEY=pk_live_...
VITE_META_PIXEL_ID=
```

The frontend normally obtains the Pixel ID and Facebook App ID through:

```
GET /api/facebook/config/
```

The Meta App Secret and Conversions API token must never be exposed to the browser.

5. Configure domains and privacy

Before public release:

- Verify the production domain in Meta Business settings
- Publish privacy and data-deletion instructions
- Explain Facebook Login and advertising measurement in the privacy policy
- Obtain consent where required
- Respect Global Privacy Control and regional consent requirements
- Do not send prohibited or unnecessary personal data
- Hash permitted customer identifiers before sending server events
- Define retention and deletion schedules

Facebook API endpoints

Read integration configuration

```
curl http://localhost:8000/api/facebook/config/
```

Verify a Facebook Login access token

```
curl -X POST http://localhost:8000/api/facebook/login/ \
-H 'Content-Type: application/json' \
-d '{"access_token":"USER_ACCESS_TOKEN"}
```

The backend validates the token against the configured Meta App and then requests the connected user profile.

List Pages managed by the organizer

```
curl -X POST http://localhost:8000/api/facebook/pages/ \
-H 'Content-Type: application/json' \
-d '{"access_token":"USER_ACCESS_TOKEN"}'
```

Generate a Group-specific share link

```
curl -X POST http://localhost:8000/api/campaigns/bring-band-x-to-new-york/facebook/share-link/ \
-H 'Content-Type: application/json' \
-d '{
  "source":"facebook_group",
  "group_name":"Band X NYC Fans",
  "referral_code":"admin-jane"
}'
```

Response:

```
{
  "campaign_url": "https://api.example.com/share/campaign/bring-band-x-new-york/?
    source=facebook_group&group=Band+X+NYC+Fans&ref=admin-jane",
  "share_dialog_url": "https://www.facebook.com/dialog/share?..."
}
```

Publish to a managed Facebook Page

```
curl -X POST http://localhost:8000/api/campaigns/bring-band-x-to-new-york/facebook/publish-page/ \
-H 'Content-Type: application/json' \
-d '{
  "page_id":"FACEBOOK_PAGE_ID",
  "page_access_token":"PAGE_ACCESS_TOKEN",
  "source":"facebook_page",
  "referral_code":"official-page",
  "message":"Should Band X play New York? Support the seed and help make the gig happen."
}'
```

The access token is used for the request and is not written into the campaign audit event.

Send a server or offline conversion

```
curl -X POST http://localhost:8000/api/campaigns/bring-band-x-to-new-york/facebook/track-conversion/ \
-H 'Content-Type: application/json' \
-d '{
  "event_name": "AttendEvent",
  "event_id": "checkin-ticket-456",
  "email": "fan@example.com",
  "action_source": "physical_store",
  "currency": "USD",
  "custom_data": {
    "event_id": "vibes-event-123",
    "ticket_id": "ticket-456"
  }
}'
```

Open Graph sharing

Facebook should share the backend URL rather than the JavaScript-only frontend URL:

```
/share/campaign/<slug>/
```

That page contains server-rendered metadata:

```
<meta property="og:type" content="website">
<meta property="og:title" content="Bring Band X to New York">
<meta property="og:description" content="500 supporters needed...">
<meta property="og:url" content="https://.../share/campaign/bring-band-x-new-york/">
<meta property="og:image" content="https://cdn.../campaign-image.jpg">
```

It also displays a lightweight campaign landing page and routes supporters to the React interface while preserving source, Group, and referral parameters.

Official sharing documentation:

- <https://developers.facebook.com/documentation/sharing/web>
- <https://developers.facebook.com/documentation/sharing/reference/share-dialog/>

Meta Pixel and Conversions API

Browser events

The frontend loads Meta Pixel only when a Pixel ID is configured.

A commitment uses the returned pledge ID as the browser event ID:

```
pledge:<pledge-uuid>:created
```

Server events

The backend sends the same event ID to Conversions API, allowing Meta to deduplicate browser and server versions of the event.

Examples:

```
Lead           attendance commitment without deposit
InitiateCheckout refundable deposit initiated/recorded
Purchase        deposit finalized after gig confirmation
AttendEvent     physical QR check-in
```

The core campaign flow never depends on Meta attribution availability. A Meta API outage must not prevent a supporter from committing, paying, receiving a refund, or entering the event.

Data minimization

The reference implementation hashes email addresses with SHA-256 before sending them to Conversions API. A production system should additionally handle consent, cookies, `_fbp`, `_fbc`, client IP, user agent, regional restrictions, and data-processing terms in accordance with applicable law and Meta requirements.

Campaign state machine

```
DRAFT
  ↓ launch
COLLECTING
  ↓ target reached
TARGET_REACHED
  ↓ artist or venue confirmation begins
CONFIRMING
  ↓ both confirmed
CONFIRMED
  ↓ deposits finalized and event created
LIVE
  ↓ event occurs
COMPLETED

COLLECTING + deadline missed → REFUNDING → REFUNDED
Any pre-live cancellation    → REFUNDING → REFUNDED
```

Run with Docker

```
cd demand-gig-mvp
docker compose up --build
```

Open:

- Frontend: <http://localhost:5173>
- API: <http://localhost:8000/api/campaigns/>
- Admin: <http://localhost:8000/admin/>

Create a demo campaign and an admin account:

```
docker compose exec backend python manage.py seed_demo
docker compose exec backend python manage.py createsuperuser
```

Apply the Facebook URL migration:

```
docker compose exec backend python manage.py migrate
```

Run without Docker

Backend:

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python manage.py migrate
python manage.py runserver
```

Frontend:

```
cd frontend
npm install
npm run dev
```

Core API examples

1. Plant the seed

```
curl -X POST http://localhost:8000/api/campaigns/ \
-H 'Content-Type: application/json' \
-d '{
  "title": "Bring Band X to New York",
  "pitch": "Prove demand before booking the artist and venue.",
  "artist_name": "Band X",
  "city": "New York",
  "country": "United States",
  "deadline": "2026-09-01T23:59:00-04:00",
  "goal_type": "both",
  "supporter_target": 500,
  "amount_target": "25000.00",
  "suggested_deposit": "25.00",
  "currency": "USD",
  "organizer_name": "Open Concert Community",
  "organizer_email": "organizer@example.com",
  "facebook_event_url": "https://www.facebook.com/events/example",
  "facebook_group_url": "https://www.facebook.com/groups/example",
  "facebook_page_url": "https://www.facebook.com/example"
}'
```

2. Launch it

```
curl -X POST http://localhost:8000/api/campaigns/bring-band-x-to-new-york/launch/ \
-H 'Content-Type: application/json' -d '{}'
```

3. Support it from a Facebook community

```
curl -X POST http://localhost:8000/api/campaigns/bring-band-x-to-new-york/pledge/ \
-H 'Content-Type: application/json' \
-d '{
  "supporter_name": "Alex Fan",
  "supporter_email": "alex@example.com",
  "quantity": 1,
  "amount": "25.00",
  "idempotency_key": "browser-generated-uuid",
  "source": "facebook_group",
  "source_label": "Band X NYC Fans",
  "referral_code": "admin-jane"
}'
```

4. Confirm artist and venue

```
curl -X POST http://localhost:8000/api/campaigns/bring-band-x-to-new-york/confirm-artist/ \
-H 'Content-Type: application/json' -d '{"details":"Signed artist LOI dated August 20"}'

curl -X POST http://localhost:8000/api/campaigns/bring-band-x-to-new-york/confirm-venue/ \
-H 'Content-Type: application/json' -d '{"details":"Venue hold and capacity approved"}'
```

5. Convert it into the confirmed event

```
curl -X POST http://localhost:8000/api/campaigns/bring-band-x-to-new-york/finalize/ \
-H 'Content-Type: application/json' -d '{"event_id":"vibesmeet-event-123"}'
```

Stripe configuration

Backend:

```
PAYMENT_PROVIDER=stripe
STRIPE_SECRET_KEY=sk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...
```

Frontend:

```
VITE_STRIPE_PUBLISHABLE_KEY=pk_test_...
```

Webhook:

```
/api/payments/stripe/webhook/
```

The implementation charges a refundable deposit rather than calling it escrow. Long campaign periods generally should not depend on card authorization holds. If the target is missed, a refund is issued to the original PaymentIntent.

Suggested VibesMeet integration boundary

Keep demand campaigns as their own bounded context and publish events when state changes occur:

```
campaign.launched
facebook.share_link_created
facebook.page_post_published
facebook.conversion_forwarded
pledge.created
sponsor.pledged
campaign.target_reached
artist.confirmed
venue.confirmed
campaign.finalized
pledge.refunded
```

VibesMeet Event OS subscribes to `campaign.finalized`, creates the normal ticketed event, and returns the event ID.

SponsorOS consumes:

- Supporter geography
- Facebook source and community attribution
- Verified supporter total
- Verified committed amount
- Sponsor commitments
- Conversion rate by Group/Page/referral code
- Confirmed ticket sales
- QR attendance
- Sponsor impressions and post-event reporting

Testing and validation

Run the dependency-free package checks:

```
python scripts/static_checks.py
```

Run the complete backend, frontend, migration, and optional Docker validation in a normal development environment:

```
./scripts/run_full_tests.sh
```

The full results, limitations, and corrections made during validation are documented in [TEST_REPORT.md](#). The combined README and product screenshots are included at [docs/Demand_Driven_Gig_MVP_README_and_Screenshots.pdf](#).

Production security and hardening

1. Add VibesMeet authentication, role-based authorization, and organizer verification.

2. Protect all Facebook Page and conversion endpoints with organizer/admin permissions.
3. Never persist user or Page access tokens in plaintext.
4. Use encrypted token storage or a secrets vault when long-lived Page access is required.
5. Rotate, revoke, and monitor Meta tokens.
6. Validate that a connected user has the necessary Page task before publishing.
7. Add CSRF, OAuth state, nonce, redirect allowlists, and session binding.
8. Run Conversions API events through an idempotent asynchronous queue.
9. Store Meta event IDs and prevent duplicate submissions.
10. Add rate limits to share-link generation, Login verification, Page publishing, and conversion endpoints.
11. Do not allow arbitrary untrusted URLs in Page posts without validation.
12. Add consent management for Pixel and advertising measurement.
13. Add a payout ledger using double-entry accounting.
14. Run refunds asynchronously with retries and dead-letter handling.
15. Add tax, refund, cancellation, chargeback, age, privacy, sanctions, and accessibility rules.
16. Have counsel approve campaign and deposit-conversion terms.
17. Do not claim funds are held in escrow unless a compliant escrow arrangement exists.
18. Add artist and venue contract upload, e-signature, insurance, permit, and capacity workflows.
19. Add fraud controls, velocity limits, bot detection, email verification, and sponsor due diligence.
20. Add metrics, tracing, backups, secret management, security scanning, and incident response.

Important product distinction

Ordinary event platforms work like this:

```
Book artist and venue → spend money → publish tickets → hope enough people attend
```

This product works like this:

```
Propose the gig → prove the audience → verify money and sponsors → confirm artist and venue → produce the event
```

That is the core Open Concert concept and the missing demand-validation layer that can sit in front of VibesMeet Event OS and SponsorOS.

Validation and Test Report

Project: Demand-Driven Gig Creation MVP
Validation date: July 29, 2026
Source tested: Uploaded demand-gig-mvp(1) (1).zip

Executive result

The package passed all dependency-free structural, syntax, document, image, configuration, and archive-integrity checks available in the execution environment. Several defects found during review were corrected before repackaging.

Full framework execution was attempted but could not be completed inside this sandbox because its Python and npm package registries do not contain the required Django, React, Vite, or Stripe packages, and Docker is not installed. The project includes `scripts/run_full_tests.sh` so the complete runtime suite can be executed in a normal internet-connected development environment.

Verified checks

Area	Result	What was verified
Uploaded ZIP	PASS	<code>unzip -t</code> reported no compressed-data errors.
Python source	PASS	Every backend <code>.py</code> file parsed and compiled successfully.
TypeScript source	PASS	All <code>.ts</code> and <code>.tsx</code> files passed TypeScript checking using local declarations for unavailable third-party packages.
JSON	PASS	<code>package.json</code> and <code>tsconfig.json</code> parsed successfully.
Docker Compose	PASS	YAML parsed; database, backend, and frontend services are present; backend waits for database health.
Migration structure	PASS	Sequential migrations 0001 through 0003; campaign-scoped pledge idempotency exists in both model and migration.
Frontend/backend API contract	PASS	Campaign, pledge, sponsor, Facebook configuration/login/Page/share endpoints match the frontend request paths and slug lookup.
Screenshots	PASS	All three required PNGs have valid PNG headers and nonzero dimensions.
Combined PDF	PASS	PDF opens, is not encrypted, has 26 pages, embedded fonts, a valid EOF marker, and rendered successfully to 26 PNG pages.

Area	Result	What was verified
Secrets scan	PASS	No live Stripe secret, webhook secret, Facebook access token, or private key was found in the packaged source.

Runtime tests attempted but environment-blocked

Test	Status	Reason
python manage.py check	BLOCKED	Django and Django REST Framework are unavailable in the sandbox registry.
Django migrations against SQLite/PostgreSQL	BLOCKED	Same missing Python dependencies; PostgreSQL service cannot be started without Docker.
Django unit tests	BLOCKED	Same missing Python dependencies.
npm install and production Vite build	BLOCKED	The sandbox npm registry returns 404 for React, Vite, TypeScript, and Stripe packages.
Docker Compose end-to-end startup	BLOCKED	Docker CLI/daemon is not installed in the sandbox.
Live Stripe webhook/payment/refund test	NOT RUN	Requires Stripe test credentials, outbound connectivity, and a webhook tunnel.
Live Meta Login/Page publishing/Conversions API test	NOT RUN	Requires a configured Meta app, approved permissions, access tokens, and outbound connectivity.

These blocked checks are not reported as passes. Run `./scripts/run_full_tests.sh` in a normal development environment to execute them.

Defects found and corrected

- Cross-campaign idempotency collision:** pledge idempotency keys were globally unique and could return a pledge from another campaign. They are now unique per campaign, with migration `0003_scope_pledge_idempotency.py`.
- Stripe retry usability:** a repeated pending Stripe request could lose its client secret. The payment provider can now retrieve it from the existing `PaymentIntent`.
- Incomplete failure cleanup:** nonfinancial attendance commitments remained active after campaign failure. They are now canceled.
- Sponsor cleanup:** pledged sponsors are canceled when a campaign fails; paid/finalized sponsor commitments enter the refund workflow when a payment reference exists.
- Sponsor finalization:** sponsor commitments are marked finalized when the campaign becomes a live event.
- Webhook state regression:** delayed Stripe events could overwrite terminal pledge states. Webhook updates now apply only while a pledge is pending.
- Campaign goal validation:** invalid supporter or financial thresholds are rejected by the API serializer before launch.
- Currency normalization and display:** currency codes are normalized to uppercase and campaign totals use currency-aware frontend formatting.

9. **Facebook readiness flag:** Facebook connection is reported as enabled only when both the App ID and App Secret are configured.

Automated backend test coverage included

The packaged Django test suite covers:

- Draft launch and collecting state
- Combined supporter/money threshold transition
- Artist and venue confirmation gates
- Final conversion into a live event
- Paid pledge refund
- Cancellation of nonfinancial attendance commitments
- Sponsor cancellation and finalization
- Campaign-scoped idempotency
- Sponsor-triggered money threshold
- Early confirmation rejection
- Expired campaign launch rejection
- Unique slug generation
- Facebook share-link encoding and attribution
- Currency normalization
- Invalid financial target rejection

Remaining production risks

This is a reference MVP, not a production authorization or payments system. Before deployment, add VibesMeet identity and role-based permissions, organizer ownership checks, rate limiting, asynchronous payment/refund jobs, durable idempotency records for Meta events, encrypted token storage, double-entry payout accounting, audit monitoring, privacy consent, and legal review of deposit/refund terms.

Reproduce the complete test suite

```
unzip demand-gig-mvp-tested.zip
cd demand-gig-mvp
./scripts/run_full_tests.sh
```

For an end-to-end Docker smoke test after the script passes:

```
docker compose up --build
# In another terminal:
curl -f http://localhost:8000/api/campaigns/
curl -f http://localhost:5173/
```